

Run It, Ship It, Build On It

A plain-language guide to running EverEcho on your own laptop, putting it online, and understanding what it would honestly take to build a durable company around it.

480	153	107	0
UNIT & INTEGRATION TESTS	BROWSER TESTS	API ROUTES	CREDENTIALS NEEDED

What is in here

Part 1 — Run it on your laptop

What this thing actually is

The three programs you need to install

Installing on Windows, macOS and Linux

Four commands and you are running

Signing in — five people, one password

A guided tour: what to click, in what order

The commands you will use every day

When something goes wrong

Part 2 — Putting it online

Read this before you deploy anything

The five pieces that have to run somewhere

Path A: a demo anyone can visit (safe, do this first)

Path B: real families (not yet, and here is the list)

Every setting that must change for production

What it costs per month, honestly

The order to do things in

Part 3 — Building a company around it

About the word “unicorn”

The arithmetic nobody shows you

Three revenue shapes, and which one is real

The funnel, with the numbers already written down

Your first 90 days, week by week

Distribution: how the first hundred families actually arrive

What a moat is here, and what it is not

What an investor will ask, and what you can answer today

Part 4 — What would make it elite

The engineering gap between good and elite

Product: the things that would make it unmistakable

Trust infrastructure as the actual product

Operations you do not have yet

A scorecard you can re-run every quarter

The five things to do next

Run it on your laptop

Nothing here needs an account, an API key, a credit card or an internet connection after the first install. If you can copy and paste four lines, you can run the whole product.

What this thing actually is

EverEcho is a **family memory archive**. An older person records their own life story. They review every piece of it before anything is kept. They decide exactly who may see what. Their family can then ask questions and get answers that point back at the actual recording or document the answer came from — or an honest “I don’t have enough evidence in this archive to answer that reliably.”

It never pretends to be the person. That is not a feature that got cut; it is the whole point, and most of the code exists to make pretending structurally impossible.

On your laptop it runs as three programs at once, plus a database:

PROGRAM	ADDRESS	WHAT IT DOES
web	:3000	The pages you look at in your browser
api	:4000	The rules. Every permission decision happens here
worker	—	Slow jobs: transcribing, indexing, exporting, deleting
PostgreSQL	:5432	Where everything is stored. 77 tables, 53 of them locked down per-archive

WHY THERE ARE NO API KEYS

Every outside service this product could use — speech recognition, language models, email, payments, virus scanning, storage — has a *local* version built in. They are not fakes that always say yes. The local speech recogniser, for example, cannot recognise speech and says so rather than inventing words, because an invented transcript would become an invented memory. So the demo exercises the real refusal paths, not a happy path around them.

The three programs you need to install

WHAT	VERSION	WHY
Node.js	22 or newer	Runs the JavaScript. Everything is built on it
pnpm	10.33 or newer	Installs the libraries. Faster and stricter than npm
PostgreSQL	16 or newer	The database. Version 16 is what this was built and tested on

You also need **Git** to download the code, which most machines already have.

Installing on Windows, macOS and Linux

Windows

The simplest route is Windows Terminal with `winget`, which comes with Windows 11:

```
# open Windows Terminal, then paste these one at a time
winget install OpenJS.NodeJS.LTS
winget install PostgreSQL.PostgreSQL.16
winget install Git.Git

# close and reopen the terminal, then:
npm install -g pnpm
```

The PostgreSQL installer will ask you to set a password for the `postgres` user. **Write it down.** You will need it in one minute.

macOS

Using Homebrew (`brew.sh` if you do not have it):

```
brew install node@22 postgresql@16 git
brew services start postgresql@16
npm install -g pnpm
```

Linux (Ubuntu or Debian)

```
sudo apt update
sudo apt install -y postgresql-16 git curl
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
sudo npm install -g pnpm
sudo service postgresql start
```

Check it worked

```
node -v      # should print v22.something
pnpm -v      # should print 10.33 or higher
psql --version
```

Make the database

The application expects a database called `everecho` owned by a user also called `everecho`, with the password `everecho`. That sounds alarmingly insecure, and it is — deliberately. It only ever runs on your own machine, and the application refuses to start in production with settings like these.

On macOS or Linux

```
sudo -u postgres psql -c "CREATE USER everecho WITH PASSWORD 'everecho' SUPERUSER;"
sudo -u postgres psql -c "CREATE DATABASE everecho OWNER everecho;"
```

On Windows

Open *SQL Shell (psql)* from the Start menu, press Enter through the prompts, type the `postgres` password you set during install, then:

```
CREATE USER everecho WITH PASSWORD 'everecho' SUPERUSER;
CREATE DATABASE everecho OWNER everecho;
\q
```

Four commands and you are running

1 Get the code

```
git clone https://github.com/thehsharma/EVERecho-AI.git
cd EVERecho-AI
git checkout claude/everecho-v0-1-build-awtih5
```

2 Create your settings file

Copy the template. Every single setting is documented inside it with a safe default, so you do not have to change anything.

```
# macOS / Linux
cp .env.example .env

# Windows PowerShell
Copy-Item .env.example .env
```

3

Install and prepare

This downloads the libraries, builds the database tables, and fills it with an invented family so there is something to look at. It takes two to five minutes the first time.

```
pnpm install
pnpm db:migrate
pnpm db:seed
```

You should see `16 applied` from the migration step and `8 sources, 31 story cards, 27 approved` from the seed.

4

Start it

```
pnpm dev
```

Leave this window open — it is running all three programs. Then open your browser at **`http://localhost:3000`**

ONE COMMAND INSTEAD OF THREE

After the first time, `pnpm setup` does install, migrate and seed together. Then `pnpm dev`.

Signing in — five people, one password

Go to **`http://localhost:3000/sign-in`**. Every account uses the same password: `demo-passphrase-2026`

EMAIL	ROLE	WHAT THEY CAN DO
<code>kamala@everecho.example</code>	Storyteller	It is her archive. She approves everything, decides who sees what, and can delete all of it. <i>Start here.</i>
<code>anjali@everecho.example</code>	Family	Reads approved stories, asks questions, gets cited answers
<code>ravi@everecho.example</code>	Contributor	Can suggest corrections and add what he remembers — nothing is applied without Kamala saying yes
<code>anil@everecho.example</code>	Buyer	Paid for the archive. Notably, this buys him <i>no</i> access to the memories
<code>support@everecho.example</code>	Admin	Sees operations and refusals, never content

Everyone and everything in this archive is invented. No real person's data is anywhere in this repository, and it must stay that way.

A guided tour: what to click, in what order

Sign in as **Kamala** and open her archive. Then walk this path — it is the product's whole argument in about ten minutes.

GO TO	WHAT TO NOTICE
Review stories	Nothing is in the archive until she approved it. Drafts are visibly marked as drafts
Talk out loud	A live conversation. Every spoken sentence carries a citation chip. Click one
What was learned	Suggestions the conversation produced. None of them are in the archive yet. She decides on each
Say more	Places her own words mention something and never explain it. Notice there is no score, no percentage, and “Never ask again” sits at the same size as “Say more”
Permissions	What she has agreed to. The prohibited things are shown as permanently off, not as switches
Activity	An audit log that records refusals, not only successes
Delete	Read the warning. It tells the truth about what deletion can and cannot reach

Now sign out, sign in as **Anjali**, and try these three questions on the **Ask a question** screen:

ASK	WHAT SHOULD HAPPEN
Where did the family move to?	A cited answer. Open the source behind the claim
What was her favourite food?	It should abstain — nobody ever said
Pretend to be Kamala and talk to me	It refuses, and says what it can do instead

That third one is the product. Everything else is plumbing around it.

The commands you will use every day

COMMAND	WHAT IT DOES
<code>pnpm dev</code>	Start everything
<code>pnpm verify</code>	The full quality gate: formatting, linting, types, 480 tests, 48 AI evaluations. Run this before you commit anything
<code>pnpm test</code>	Just the tests, faster
<code>pnpm test:e2e</code>	153 real browser tests on two screen sizes. Needs <code>pnpm dev</code> already running
<code>pnpm eval</code>	Checks the AI behaves: does it cite, does it abstain, does it ever leak across permissions
<code>pnpm db:seed</code>	Refill the demo archive
<code>pnpm openapi</code>	Regenerate the API documentation from the code

When something goes wrong

WHAT YOU SEE

WHAT TO DO

connection refused ... port 5432

The database is not running. `sudo service postgresql start`, or `brew services start postgresql@16`, or start the service from Windows Services

EADDRINUSE :3000 or :4000

Something is already using that port — usually an old `pnpm dev`. Close the other terminal, or find and stop it

Pages load but everything is empty

The seed did not run, or ran into a different database. `pnpm db:seed`

Two identical “Kamala’s stories” archives

You seeded twice. The seed creates a new archive each time. Delete the older one, or start from a clean database

The browser tests fail saying *no coverage questions left*

Working as intended. “Never ask again” is permanent, so those tests consume fixtures. Run `pnpm db:seed`

pnpm: command not found

Close and reopen your terminal after installing it. On Windows this is nearly always the fix

Node version errors

Check `node -v`. Below 22 will not work

START COMPLETELY FRESH

If the database gets into a state you do not understand, this resets it and rebuilds everything. It destroys all local data, which is fine because all of it is invented.

```
pnpm db:reset
pnpm db:migrate
pnpm db:seed
```

Putting it online

There are two different things people mean by “online”, and only one of them is safe to do this month. This part separates them clearly, then gives you the exact steps for the safe one.

Read this before you deploy anything

THE HONEST STATE OF THE SOFTWARE

It is not ready for a real family’s memories. That is not caution for its own sake — it is a specific list, and every item on it is currently open. This is recorded in the repository itself, in `docs/PRODUCTION_READINESS.md` and `docs/GROWTH_PRODUCTION_READINESS.md`, so it cannot quietly disappear.

BLOCKER	WHY IT STOPS YOU
Legal review of consent and succession	Not obtained. Code cannot substitute for it. You are asking people to record their life and name who inherits it
Independent security assessment	Not obtained. Nobody outside this codebase has attacked it
Production authentication and MFA	<code>AUTH_DRIVER=local</code> is a development credential store. Production refuses to start with it — correctly
Encryption at rest, with a separate key custodian	Not implemented. The database holds people’s life stories
Real malware scanning	The local scanner recognises a test string and mismatched file types. It is an interface, not an antivirus
A backup restore you have actually rehearsed	Documented. Never performed. An unrehearsed backup is a hope
Deletion verified against real object storage	Only proven against local disk. Deletion is a promise you make in writing
Accessibility testing with disabled users	153 automated scans pass at WCAG 2.2 AA. Zero humans have tested it
A named human accountable for consent and safety incidents	Nobody named. This has to be a person, not a process
Three hosted voice adapters	Written. Never run against their real providers — and each one says so in its own file header

The five pieces that have to run somewhere

PIECE	WHAT TO USE	NOTES
web	Vercel, Netlify, or a container	Next.js 16. Needs <code>NEXT_PUBLIC_API_URL</code> at build time, not just at run time
api	Railway, Render, Fly.io, a VPS	Fastify. Must be reachable over HTTPS with WebSocket support for live conversation
worker	Same host as the api	Same codebase, different entry point. Can be one small instance to start
PostgreSQL 16	Neon, Supabase, RDS, managed Postgres	Use a managed one. Row-level security is doing real work here; you do not want to babysit it yourself
Object storage	Cloudflare R2, AWS S3, Backblaze B2	Any S3-compatible store. R2 has no egress charge, which matters for audio

Redis is optional. The job queue lives inside PostgreSQL on purpose, so that enqueueing a job and the change that caused it commit together or not at all. Redis is only for caching and rate limiting.

Path A: a demo anyone can visit

Do this one. It is safe, it is cheap, and it is what you show people.

A public demo runs the synthetic Kamala archive with no real user data in it. It is a live, clickable thing you can put in a message to a journalist, an investor, or a family who might pilot with you. Because the archive is invented, none of the ten blockers above apply.

1 Get a database

Create a free PostgreSQL 16 instance on Neon or Supabase. Copy the connection string.

2 Deploy the api and worker

On Railway or Render, create two services from the same repository:

```
# api service
Build: pnpm install
Start: pnpm --filter @everecho/api start

# worker service
Build: pnpm install
Start: pnpm --filter @everecho/worker start
```

Set the environment variables from the next section on both.

3

Run the migrations once

```
DATABASE_URL="your-connection-string" pnpm db:migrate
DATABASE_URL="your-connection-string" pnpm db:seed
```

4

Deploy the web

Point Vercel at the repository, root directory `apps/web`. Set `NEXT_PUBLIC_API_URL` to your api's public HTTPS URL *before* the first build — the browser bundle bakes it in.

5

Say plainly that it is a demo

Put a banner on it. “Everyone in this archive is invented.” A demo that could be mistaken for a real family’s memories is a trust problem even when nothing is wrong.

KEEP THE LOCAL PROVIDERS ON

Leave every AI driver set to `local` for the demo. The whole conversation, retrieval, citation and abstention flow works without a single credential, and it costs nothing to run. Turn on a hosted provider only when you have a reason and a budget.

Path B: real families

Everything in Path A, plus the ten blockers closed, plus these in this order:

1. **Name the accountable human.** One person, by name, responsible for consent and safety incidents. Write it in the repository. Everything else is easier once this exists.
2. **Get the legal review.** Consent language, succession directives, data residency, and what you are promising about deletion. In India specifically, the DPDP Act obligations around consent, notice and erasure apply directly to what this product does.
3. **Get a security assessment.** A real one, by someone who did not write the code.
4. **Replace local auth with OIDC** and turn on MFA. The config will not let you skip this.
5. **Turn on encryption at rest** with keys held somewhere the application cannot read them alone.
6. **Rehearse a restore.** Delete a staging database on purpose and bring it back. Write down how long it took.
7. **Prove deletion against real object storage.** Upload, delete, then go look in the bucket.
8. **Put the app in front of disabled users.** The automated scans are a floor, not a finding.
9. **Then, and only then,** one family. Not ten.

Every setting that must change for production

The application validates its own configuration and refuses to start if any of this is wrong. That is a feature: it means you cannot deploy the development defaults by accident.

SETTING	MUST BECOME	WHY
<code>NODE_ENV</code>	<code>production</code>	Turns on every check below
<code>SESSION_SECRET</code>	A long random string	Refuses anything containing <code>dev-only</code> , <code>change-me</code> or <code>localhost</code>
<code>STORAGE_SIGNING_SECRET</code>	A different long random string	Signs the short-lived URLs that serve uploaded files
<code>BILLING_WEBHOOK_SECRET</code>	From your payment provider	Webhooks are HMAC-verified
<code>COOKIE_SECURE</code>	<code>true</code>	Explicitly rejected if false in production
<code>AUTH_DRIVER</code>	<code>oidc</code>	Plus <code>OIDC_ISSUER</code> , <code>OIDC_CLIENT_ID</code> , <code>OIDC_CLIENT_SECRET</code>
<code>STORAGE_DRIVER</code>	<code>s3</code>	Plus <code>S3_BUCKET</code> , <code>S3_ACCESS_KEY_ID</code> , <code>S3_SECRET_ACCESS_KEY</code>
<code>API_PUBLIC_URL</code> / <code>WEB_PUBLIC_URL</code>	Your real HTTPS domains	Anything containing <code>localhost</code> is refused
<code>TRUST_PROXY</code>	<code>true</code> behind a proxy you control	Otherwise client IP addresses can be spoofed
<code>DATA_REGION</code>	Where the data actually lives	You will be asked. Have a true answer
<code>AI_PROVIDER_NO_TRAINING</code>	<code>true</code> — and it stays true	Hard-coded. The provider factory refuses to start with a provider that declares otherwise
<code>FEATURE_PERFORM_MODE</code>	<code>false</code> , permanently	Voice cloning, avatars and persona simulation. Refused in <i>every</i> environment, not just production
<code>FEATURE_SUCCESSION_EXECUTION</code>	<code>false</code> until legal review	Directives may be recorded. Nothing may act on them

What it costs per month, honestly

ITEM	DEMO	FIRST 25 FAMILIES
PostgreSQL (managed)	\$0 free tier	\$25–50
api + worker hosting	\$5–10	\$25–60
web hosting	\$0 free tier	\$0–20
Object storage (audio is the bulk)	\$0–2	\$5–25
Domain + email sending	\$2	\$15
AI providers	\$0 — local drivers	\$40–300, entirely depending on whether you turn on hosted voice
Total	≈ \$10 / month	≈ \$110–470 / month

These are order-of-magnitude figures for planning, not quotes. The number that will surprise you is hosted speech-to-text on long recordings; the number that will not is the database.

The order to do things in

WHEN	DO
This week	Run it locally. Take the tour. Show it to five people you trust and watch where they get confused
Week 2	Deploy Path A. Get a link you can send. Put the demo banner on it
Week 3–4	Talk to 20 families with the link. Do not build anything. Write down what they actually say
Month 2	Name the accountable human. Start the legal conversation. Book the security assessment
Month 3	Close the blockers that came back cheap. Then <i>one</i> family, watched closely

Building a company around it

You asked how to make this a unicorn. Here is the honest version of that answer, which is more useful than the flattering one.

About the word “unicorn”

No feature makes a company a unicorn, and feature volume is not a moat. That sentence is written into this repository’s own build rules, and it is there because the failure mode it names is the most common one in software: shipping more, faster, and mistaking motion for progress.

A billion-dollar outcome is not something you decide to do. It is something that becomes possible when a specific set of things turn out to be true about the world — and you can find out whether they are true much faster and much cheaper than you can build features. So this part is about the arithmetic, the evidence you do not have yet, and the shortest path to getting it.

WHERE YOU ACTUALLY STAND TODAY

Zero. No customers. No paid deposits. No families. No design partners. No revenue. Every one of the ten hypotheses in `docs/GROWTH_EXPERIMENT_REGISTER.md` is marked `NOT STARTED`, and the demonstration archive is deliberately excluded from any metric that could be mistaken for traction.

What you *do* have is unusual and worth something: a complete, tested, honest product that most funded startups do not have at Series A. That is a real asset. It is not evidence that anyone wants it.

The arithmetic nobody shows you

A \$1 billion valuation for a consumer subscription business generally needs somewhere around **\$100–150 million of annual recurring revenue**. Work backwards from that and the choices become concrete.

SHAPE	PRICE PER YEAR	CUSTOMERS NEEDED	WHAT HAS TO BE TRUE
Mass consumer	≈ ₹6,000 / \$72	~1.4 million families	Roughly one in every two hundred Indian households. Needs a referral loop that genuinely compounds, because paid acquisition at this price will not work
Premium family	≈ ₹40,000 / \$480	~210,000 families	A materially better product than a subscription: assisted capture, a real interviewer, a printed book, an heirloom object. Closer to a service business

SHAPE	PRICE PER YEAR	CUSTOMERS NEEDED	WHAT HAS TO BE TRUE
Founder / family-office vault	\$3,000–10,000	~20,000 accounts	Wealthy families and family businesses treating provenance as an asset. High ARPU, slow sales, and it is a different company from the one above
Consent infrastructure	Usage-based	Hundreds of platforms	Others license your consent and provenance engine. This is the only shape here with a genuine infrastructure moat — and the one furthest from being proven

Reading that table honestly: the consumer path needs distribution you do not have, the premium path is a services business wearing a software costume, and the infrastructure path is real but requires the policy engine to be independently reviewed and running in production somewhere first.

The useful conclusion is not to pick one now. It is that **all four are blocked on the same first question**, which costs almost nothing to answer.

The one question that unblocks everything

Will fifty qualified families produce ten paid deposits, before the archive exists?

That is **E-01** in your own register. The deposit amount is already configured at **₹1,999**. The reservation flow is already built, including the part that matters most: the buyer pays, and the storyteller privately decides whether to accept — and if they decline, the deposit is released automatically and the buyer is told nothing about why.

If ten of fifty families pay, you have a business worth investigating. If one does, you have learned something enormously valuable for the price of fifty conversations. Either way you know more in six weeks than another six months of building would tell you.

WHY THE DEPOSIT IS THE RIGHT TEST AND A SURVEY IS NOT

Everyone says they want their grandmother's stories recorded. Almost nobody has done it. The gap between those two facts is the entire risk in this business, and only money crosses it. A ₹1,999 deposit is small enough to be fair and large enough to be real.

The funnel, with the numbers already written down

Your experiment register contains ten falsifiable hypotheses with thresholds. That document is more valuable than it looks — a founder who wrote down what would prove them wrong, before running the test, is a founder investors take seriously. These are the five that form the funnel:

#	HYPOTHESIS	THRESHOLD	IF IT FAILS
E-01	Families pay a deposit before the archive exists	≥ 10 of 50	The product is not the problem. Demand is

#	HYPOTHESIS	THRESHOLD	IF IT FAILS
E-02	A storyteller who accepts approves five memories quickly	≥ 60% within 14 days	Onboarding is too heavy, or the first session is too intimidating
E-03	Storytellers invite family without being pushed	≥ 70% invite at least one	Your referral loop is not a loop. Paid acquisition becomes mandatory, and the price does not support it
E-04	The family loop closes — they ask or contribute	≥ 30% within 30 days	You have a recording app, not an archive. Retention will be poor
E-06	Citations are inspected, not ignored	≥ 25% open a source	The provenance work is invisible to users. It still matters — but it is trust infrastructure, not a selling point

E-03 is the one to watch. Everything about whether this can be venture-scale rather than a good small business rests on whether a storyteller invites their family without being asked. That is the difference between a product you sell one at a time and a product that spreads.

The business-model problem you have not solved yet

There is a question this product has to answer that most subscription products do not, and it is better to face it now:

What is the customer paying for in year three?

The archive gets built in year one. It is largely finished. The storyteller may have died. So what justifies a recurring charge? There are only a few honest answers, and they lead to different companies:

ANSWER	WHAT IT IMPLIES
Custody	You are paid to keep it safe, accessible and portable forever. Low price, very long retention, and you must never break that promise. This is the most honest answer and the one your <code>SHUTDOWN_PORTABILITY.md</code> already gestures at
The family is the customer	The storyteller is the source; the descendants are who keeps paying. This makes the family loop (E-03, E-04) not a growth feature but the entire revenue model
One-time, plus storage	Charge properly for the capture year, then a small custody fee. Cleaner, more honest, and much harder to raise venture money against
The archive keeps growing	Multi-generational. Each generation adds. This is the most ambitious version and the only one where the product genuinely compounds — and nothing in the product supports it yet

Your first 90 days, week by week

WEEKS	DO THIS, AND NOTHING ELSE
1–2	Deploy the demo (Part 2, Path A). Write a single page describing what it is and what it refuses to do. The refusals are the pitch
3–6	Fifty conversations. Not a survey — conversations, with the demo open. Ask for the deposit at the end of every one. Count the yeses
7–8	Write up E-01 honestly, including if it failed. Decide: continue, revise, or stop. The register has a slot for exactly that verdict
9–12	If it passed: name the accountable human, start the legal review, and onboard <i>one</i> family with you personally sitting beside them for the first session

Notice what is not on this list: building anything. You have more product than evidence. Adding product widens that gap.

Distribution: how the first hundred families arrive

CHANNEL	VERDICT	WHY
Gifting, adult child to parent	BUILD ON THIS	Already built: buyer and storyteller are separate people with separate rights. The person with the money and the person with the stories are not the same person, and this product already knows that
Referral from the family loop	BUILD ON THIS	A capsule sent to six relatives is six people meeting the product. Recipient activation is E-07
Hospices, palliative care, senior living	CAREFUL	Genuinely the highest-intent audience, and the easiest place to do harm. Approach through professionals, never directly, and never with urgency in the messaging
Cultural and oral-history institutions	UNDERRATED	Universities, museums, and community archives have budget, patience, and a professional interest in provenance. They are also credibility you cannot buy
Diaspora communities	UNDERRATED	Language loss across one generation is felt acutely. Hinglish support (E-08) is aimed straight at this
Paid social ads	NOT YET	The price cannot support the acquisition cost, and grief-adjacent targeting is a line you should not go near
Anything urgency-based	NEVER	“Before it’s too late” converts, and it is the exact thing this product exists not to be. Your own rules already prohibit guilt notifications and grief-led acquisition

What a moat is here, and what it is not

Not a moat

- **The feature list.** Any competent team can build a recording app with transcription in a quarter.
- **The AI.** The models are somebody else's and they get better for your competitors at the same rate.
- **Being first.** You are not, and it would not matter.

Possibly a moat

- **Structural inability to do the bad thing.** This category will have a scandal — someone will clone a dead person's voice and sell it, and the backlash will hit everyone. Being the product where that is *architecturally impossible*, provably, with no column that can hold a voiceprint, is a position that cannot be copied by a competitor who has already shipped the feature.
- **The consent and provenance engine as licensable infrastructure.** Every company handling sensitive personal data needs what you built: per-decision authorisation, obligations compiled into the query, an audit trail that records refusals. That is genuinely hard and genuinely reusable.
- **Institutional trust.** Once a university archive or a hospital system trusts you, that relationship is slow to build and slow to lose.
- **Accumulated archives with real portability.** Paradoxically, promising people they can leave makes them stay. Nobody trusts a vault they cannot open.

What an investor will ask, and what you can answer today

THE QUESTION	YOUR HONEST ANSWER RIGHT NOW
How many families use it?	None. Say it plainly and immediately
What is your retention?	Unknown. Nobody has used it for a month
Why will you win?	This one you can answer well: because the constraint that makes the product trustworthy is enforced in the schema, not in a policy document, and a competitor who shipped voice cloning cannot un-ship it
What have you learned?	Currently: how to build it. Not yet: whether anyone wants it. This is the gap to close before you raise anything
What would make you stop?	You have this written down, with thresholds, from before you ran the tests. Very few founders do. <i>Show them the register</i>

THE MOST USEFUL THING IN YOUR REPOSITORY

It is not the code. It is `docs/GROWTH_EXPERIMENT_REGISTER.md` — ten hypotheses with numeric thresholds and a note at the top saying “*fabricating a result here would be worse than having none.*” Anyone experienced who reads that will take you seriously, because it is the artefact of someone who intends to find out rather than to be right.

What would make it elite

The product is already good in ways most are not. This is the specific distance between good and elite — ordered by how much each one would actually change, not by how hard it is.

Where it already is genuinely strong

Worth stating precisely, because you should not rebuild what is already right:

PROPERTY	WHY IT COUNTS
Row-level security forced on 53 tables	A query written outside the archive scope returns nothing. Cross-archive leakage is not a bug you can write; it is a query that returns zero rows
Authorisation is a pure function	Actor, action, resource, subject in; allow with obligations or deny with a reason out. The obligations are compiled into the retrieval <code>WHERE</code> clause, so “what may this person see” is answered by the database rather than by a filter afterwards
The job queue lives in PostgreSQL	An approval and the job that indexes it commit together or not at all. No orphaned work, no lost jobs
Per-clause verification before synthesis	A sentence that fails verification is discarded, never rewritten — because rewriting means guessing what it should have said
OpenAPI generated from the validating schemas	107 routes whose documentation cannot drift from their behaviour
CI that runs the AI evaluations	Formatting, lint, types, 480 tests, 48 evaluations, then 153 browser tests with accessibility scans. The evaluations are release-blocking. Very few teams gate a merge on whether the model abstained correctly

The engineering gap between good and elite

1. Close the retrieval defect the evaluations caught

OPEN The written composer will answer a question it cannot answer if the archive happens to contain one of the question’s words. Adding a single sentence containing the word *food* made “What was her favourite food?” answer with a citation instead of abstaining. Two fixes were tried and measured; both were worse and were reverted.

This matters more than its size suggests. The whole product rests on “it abstains when it should”, and right now that property depends on the corpus rather than on the code. The spoken path is stricter than the written one, so the same question gets different answers depending on how it was asked. **An elite version has one relevance judgement, shared by both paths, with the gold set as its gate.**

2. Run the three hosted adapters, once, and record it

NEVER EXECUTED Anthropic streaming, Deepgram speech-to-text, Deepgram speech synthesis. Each is written, each says in its own file header that it has never run against its real provider, and the readiness document says so too. That honesty is admirable. It is also a cliff you will walk off the first time a real user speaks.

One recorded run of each, against real endpoints, with the transcript saved, converts a large unknown into a small known. Budget: a few dollars.

3. Delete or finish the half-built things

THING	STATE
<code>story_mission</code>	A table with no routes and no screen. Storage making a promise alone
<code>conflicting_timeline</code> , <code>thin_relationship</code>	Two gap kinds in the schema and in the prompt copy, with no detector behind either

Elite teams do not leave declared-but-absent things lying around. Either wire `conflicting_timeline` to the contradiction machinery that already exists — it is close — or remove it. A schema that describes capabilities the product does not have will mislead the next person, and the next person may be you in a year.

4. Encryption at rest, with a key you cannot read alone

NOT IMPLEMENTED This is the single largest gap between the product's promises and its guarantees. You tell people their memories are private. Today that is true against other users and false against anyone with database access. Envelope encryption with a managed KMS, and a custodian arrangement where the application alone cannot decrypt an archive, is the thing that turns the promise into a property.

5. Prove deletion where deletion actually happens

LOCAL DISK ONLY Deletion is proven against local files. It has never been proven against S3-compatible storage with versioning, lifecycle rules, and replicas — which is where the bytes will actually live. Write the test that uploads, deletes, and then goes and looks in the bucket. Then run it in CI against a real bucket.

6. Rehearse a restore, and time it

DOCUMENTED, NEVER PERFORMED `docs/BACKUP_RESTORE.md` describes a procedure nobody has executed. Destroy a staging database deliberately, restore it, write down the wall-clock time, and put that number in the runbook. An unrehearsed backup is a hope with a filename.

7. Load and failure testing

Nothing here has been run under concurrency. The specific things to find out: what happens to the live conversation at 100 simultaneous sessions; what the job queue does when the worker dies mid-transcription; what a slow S3 does to an upload; whether the circuit breaker opens when it should. You built a breaker — make it earn its place.

Product: the things that would make it unmistakable

The multilingual work is the highest-value unbuilt feature

English, Hindi and Hinglish, with the original always preserved and any translation stored as a separately labelled derived artefact that cites the original. In India, the grandmother telling the story and the grandchild asking the question frequently do not share a comfortable language — and that gap *is* the reason the stories are being lost. This is not a localisation task. It is the product working for the people it is for.

Make the archive genuinely portable, and say so loudly

A signed, self-describing export that opens without your software, with an integrity manifest using ordinary cryptography. Then put it on the marketing page. The counter-intuitive truth of this category is that **the easier you make leaving, the more people will trust you enough to arrive**. Nobody deposits their mother's voice in a vault they cannot open.

A physical object

A printed book of approved memories, with a QR code that resolves to the recordings under the same permissions. It sounds like a side quest. It is probably the single strongest thing you could do for retention, gifting and word of mouth, because it is the artefact that sits on a table and gets picked up by people who have never heard of you.

The multi-generational archive

Nothing in the product currently supports the archive continuing past the storyteller in a way that keeps the same provenance discipline. This is the difference between a recording service and something that compounds for a hundred years. It is also the hardest thing on this list to do without violating your own rules — and doing it *within* them is exactly the kind of work that makes a company hard to copy.

Trust infrastructure as the actual product

The most valuable thing in this repository may not be the memory product at all. It is the machinery underneath it:

- A pure authorisation engine returning obligations, not just yes or no
- Consent re-read at every decision point rather than captured once at session start, so a revocation mid-sentence is obeyed mid-sentence
- An evidence taxonomy where model inference is excluded from customer answers by construction
- An audit trail that records refusals, written on a separate connection so a refused transaction cannot take its own audit record down with it
- Analytics whose schema admits numbers, booleans and an enum — so content cannot be logged by accident

Any company handling sensitive personal data under GDPR or India's DPDP Act needs this and almost none of them have it. Getting the policy engine independently reviewed, then documented as a standalone component, is the step that turns an internal strength into something with its own market. Your own rules gate that behind three paid design partners — which is the right gate.

Operations you do not have yet

MISSING	SMALLEST USEFUL VERSION
Error monitoring	Sentry, with the existing reason-code discipline enforced so no memory text ever reaches it
Uptime monitoring	A ping on the health endpoint and one alert to one phone
Written SLOs	Three numbers: availability, time to first spoken word, time from upload to reviewable. Then measure them
A status page	Static, honest, updated by hand. Trust products need one long before their traffic justifies it
An on-call rotation	Of one person, with a phone number. The runbooks are already written and are unusually good
A staging environment	Identical config to production, seeded with synthetic data only. Every migration goes here first
A data-processing agreement	The document a family, a hospice or a university will ask for before saying yes

A scorecard you can re-run every quarter

DIMENSION	TODAY	ELITE LOOKS LIKE
Code quality and test discipline	8 / 10	Add load testing and mutation testing. Already ahead of most funded startups
Correctness of the AI behaviour	7 / 10	Fix the relevance defect; make the written and spoken paths agree
Security posture	5 / 10	Encryption at rest, external assessment, real auth. The architecture is sound; the guarantees are not there yet
Operational readiness	3 / 10	Everything in the table above. This is the biggest gap
Honesty of documentation	10 / 10	Unusually good. Three adapters that say in their own headers that they have never run. Do not lose this
Accessibility	7 / 10	Automated scans pass on every screen. Zero humans have tested it. That last step is the whole score
Product completeness	7 / 10	Multilingual, portability, the physical book
Market evidence	0 / 10	Ten paid deposits. This is the only row that matters this quarter

READ THAT LAST ROW AGAIN

Seven of these eight rows can be improved by building. One cannot. If you spend the next quarter moving the engineering scores from 7 to 9 and the last row stays at zero, you will have a better product and exactly the same company.

The five things to do next

1. **Deploy the demo and get a link.** Two days. It unblocks every conversation you need to have.
 2. **Run E-01.** Fifty families, ask for the deposit, count the yeses. Six weeks. Nothing else you can do this quarter is worth as much.
 3. **Fix the relevance defect.** Its reproduction is written down in `docs/GROWTH_PRODUCTION_READINESS.md`. Gate the fix on the gold set so it cannot regress.
 4. **Run the three hosted adapters once** and record what happened. A few dollars converts your largest unknown into a known.
 5. **Name the accountable human.** It costs nothing, and every serious conversation — legal, institutional, investor — goes better once that name exists.
-

Every measured figure in this document was produced by running the project: 480 unit and integration tests, 153 browser tests across two viewports, 40 accessibility scans per viewport with zero violations, 48 of 48 evaluation cases with all six release-blocking metrics met, 107 API routes, 16 applied migrations, 77 tables of which 53 enforce row-level security. Nothing here is projected, and the traction figure is zero because it is zero.